
Algorithm 2 H₂ Eviction Algorithm, Query matrix $Q \in \mathbb{R}^{n \times d}$, key matrix $K \in \mathbb{R}^{n \times d}$, budget size of KV cache $k \in \mathbb{N}$. Formal and detailed version of Algorithm 1.

```

1: procedure H2_EVICTON( $Q \in \mathbb{R}^{n \times d}, K \in \mathbb{R}^{n \times d}, k \in \mathbb{N}$ )
2:    $S_0 \leftarrow \emptyset, \tilde{o}_0 \leftarrow 0$  ▷ Initialization the algorithm
3:   for  $i = 1 \rightarrow n$  do ▷ Linear scan each token
4:     if  $i \leq k$  then ▷ If the KV cache is not full
5:        $S_i \leftarrow S_{i-1} \cup \{i\}$  ▷ Expand the set directly
6:     else ▷ If the KV cache is full
7:        $D_i \leftarrow (\exp(Q_{i,*}(K_{S_{i-1},*})^\top) - \mathbf{1}_{[i] \setminus S_{i-1}}) \cdot \mathbf{1}_i$  ▷ Compute the normalization factor
8:        $o_i \leftarrow D_i^{-1} \cdot \exp(Q_{i,*}(K_{S_{i-1},*})^\top)$  ▷ Compute score vector
9:        $\tilde{o}_i \leftarrow \tilde{o}_{i-1} + o_i$  ▷ Accumulate the scores (Remark D.15)
10:      Let score function be  $F_{\text{score}}(T) := h(\sum_{s \in T} \tilde{o}_{i,s})$  ▷ Score function (Remark D.14)
11:       $u \leftarrow \arg \max_{v \in (S_{i-1} \cup \{i\}) \setminus \{v\}} F_{\text{score}}(S_{i-1} \cup \{i\} \setminus \{v\})$  ▷ Find an entry to swap
12:       $S_i \leftarrow (S_{i-1} \cup \{i\}) \setminus \{u\}$  ▷ Construct  $S_i$ 
13:     end if
14:   end for
15: end procedure

```

Remark D.14. We remark the above function F_{score} can have multiple choices. In the later analysis, we provide two choices. If we use the exact function, then F_{score} is $f_{S_i,i}$ (see Definition D.23 and Definition D.25). If we use the approximate function, then F_{score} is $\tilde{f}_{S_i,i}$ (see Definition D.24 and Definition D.26). Let $h : \mathbb{R} \rightarrow \mathbb{R}$ (the h being used Line 10 in Algorithm 2) denote function. We hope to choose h such that it gives the submodular property for the function F_{score} in the sense of our dynamic framework. For example, $h(z)$ is usually chosen to be some non-decreasing concave function such as $\sqrt{z+1}$ and $\log(z+1)$. For simplicity, in Algorithm 1 (which is the informal version of Algorithm 2), we choose $h(z) = z$ for the purpose of providing readers with an intuitive understanding and due to space limitation.

Remark D.15. We remark that, in Line 10 in Algorithm 2, the $\tilde{o}_{i,s}$ is the accumulation of attention score for the token s in set T . In our Algorithm 1, we only use o_s in Line 10 (see Algorithm 1) to represent that accumulation of the attention score for simplicity.

In Algorithm 2, for simplicity of the presentation, we can create multiple vectors $\tilde{o}_0, \dots, \tilde{o}_n$ (see Line 2, Line 9, Line 10). However, every time at i -th, we only need to use the information for $\tilde{o}_i$ which has at most length n size. Thus, a straightforward and better implementation for only using one length- n to store accumulative score can reduce n^2 usage to n .

D.7 Explaining Submodular Diminishing Return Property in Attention Scheme

In the standard submodular problem [123], we are given a ground set $[n]$ and a function $f : 2^{[n]} \rightarrow \mathbb{R}$. The goal is to find a set of elements S such that $f(S)$ is maximized.

We say function f is submodular (Recall the formal definition in Definition D.1), if for every $X, Y \subset [n]$ with $X \subseteq Y$ and every $x \in [n] \setminus Y$ we have

$$f(X \cup \{x\}) - f(X) \geq f(Y \cup \{x\}) - f(Y)$$

Submodular functions can represent the cost of items, due to their diminishing returns property [127, 128]. It suggests that the increase in information obtained from selecting a candidate object, such as a word or sentence, becomes smaller as more objects have already been chosen for the summary.

For instance, we introduce a new token, denoted as “w”, into two sets, S and S_0 , where the concepts covered by S_0 are a subset of those covered by S . By intuition, the information added to S_0 by “w” should be larger compared to adding it to S , as the new concepts carried by “w” might have already been covered by the concepts present in S but not in S_0 . This property is known as the diminishing return property. Hence, we propose that the neural coverage function [129] should exhibit a desirable characteristic called submodularity.

D.8 Submodular: High Level Ideas

We formalize the submodular function maximization problem with cardinality constraint in this section. Informally, a set function is submodular if it has decreasing marginal increment.

Definition D.16 (Submodular function). We denote $f : 2^{[n]} \rightarrow \mathbb{R}$ as a set function. The discrete derivative Δ_f is defined as follows:

$$\Delta_f(i | S) := f(S \cup \{i\}) - f(S).$$

A function f is submodular if, for any $S \subseteq T$ and $i \in [n] \setminus T$, the following inequality holds:

$$\Delta_f(i | T) \leq \Delta_f(i | S).$$

For simplicity, we present the problem of maximizing a submodular function with a cardinality constraint (1). Our goal is to solve the optimization problem efficiently.

$$\begin{aligned} \max_{S \subseteq [n]} \quad & f(S) \\ \text{s.t.} \quad & |S| \leq k \end{aligned} \tag{1}$$

Representation of $f(S)$ One challenge in designing the algorithm is determining the representation of input instances. Since the constraint in optimization problem (1) is straightforward, we need to decide how to represent $f(S)$. Suppose $S = i_1, i_2, \dots, i_m \subseteq [n]$, we can decompose $f(S)$ into a sum of increments as follows:

$$f(S) = f(S_0) + \sum_{j=1}^m (f(S_j) - f(S_{j-1})), \tag{2}$$

where $S_0 = \emptyset$ and $S_j = S_{j-1} \cup i_j$. Without loss of generality, we assume $f(\emptyset) = 0$. By the definition of $\Delta_f(i | S)$, we have $f(S_j) - f(S_{j-1}) = \Delta_f(i_j | S_{j-1})$. Therefore, the decomposition (2) can be simplified as follows:

$$f(S) = \sum_{j=1}^m \Delta_f(i_j | S_{j-1}) \tag{3}$$

To introduce our advanced data structure later, we further represent $\Delta_f(i | S)$ in the form of

$$\Delta_f(i | S) = u_i^\top h(S) u_i \tag{4}$$

where $u_i \in \mathbb{R}^d$ is a d -dimensional vector and $h(S) \in \mathbb{R}^{d \times d}$ is a d -by- d matrix.

In practice, a significant subclass of submodular functions is the monotone submodular functions, i.e. functions f satisfying $f(A) \leq f(B)$ for all $A \subseteq B \subseteq [n]$. When f is monotone, we could restrict all $h(S)$ to be positive semidefinite (PSD) matrixes. When the matrix $h(S)$ is positive semidefinite (PSD), it makes us achieve a faster acceleration.

D.9 Robust Greedy with Error Propagation

The procedure for the greedy selection algorithm initiates with empty set $S_0 = \emptyset$. For each iteration in the main loop, the algorithm chooses the element that maximizes the marginal increment to add to the set. When the size of the set eventually reaches k , the algorithm return that set S . Specifically, for iteration $t \in \{1, 2, \dots, k\}$, we let

$$S_t \leftarrow S_{t-1} \cup \{j_t\}$$

where the element in the singleton is $j_t = \arg \max_{j \in \bar{S}_{t-1}} f(S_{t-1} \cup \{j\})$. The greedy strategy is effective in the sense that the approximation error of it is $1 - 1/e$.

Theorem D.17 ([130]). For a monotone submodular function f , the greedy algorithm (Algorithm 3) guarantees to output a set S satisfying

$$f(S) \geq (1 - 1/e) \max_{|T|=k} \{f(T)\}.$$

Corollary D.18 ([131]). Given

Algorithm 3 Greedy algorithm benchmark

```

1: procedure GREEDYALGORITHM(submodular function  $f$ )
2:    $S_0 \leftarrow \emptyset$  ▷ Initialize an empty set  $S_0$ 
3:   for  $t = 0 \rightarrow k - 1$  do
4:      $j \leftarrow \arg \max_i \{f(S_t \cup \{i\})\}$  ▷ Find the element  $i \in [n]$  that maximize  $f(S_t \cup \{i\})$ 
5:      $S_{t+1} \leftarrow S_t \cup \{j\}$  ▷ Append element  $i$  to set  $S_t$  to get  $S_{t+1}$ 
6:   end for
7:   return  $S_k$ 
8: end procedure

```

- accuracy parameter $\epsilon > 0$.
- integer $k \geq 1$
- Let O denote an oracle that takes an arbitrary set $S \subseteq [n]$ and $i \in [n] \setminus S$, returns a value $O(S, i)$ with guarantee that $\Delta(i|S) - \epsilon \leq O(S, i) \leq \Delta(i|S) + \epsilon$.
- Let A denote an algorithm that each time step $t = 1, 2, \dots, k$, it selects $j_t = \arg \max_j \{O(S_{t-1}, j)\}$ and lets $S_t \leftarrow S_{t-1} \cup \{j_t\}$.

Then this algorithm A

- returns a set S_k
- the S_k satisfy that

$$f(S_k) \geq (1 - 1/e) \max_{|T|=k} \{f(T)\} - k(2 - 1/e)\epsilon.$$

D.10 Robust Submodular and Adding Items

We first propose a lemma that shows the robustness of the submodular.

Lemma D.19. Suppose that for each $i \in [n]$, we have

- for all $S_i \subset [n]$ with $|S_i| \leq k$
- for all $X \subset [n]$, $X \subseteq S_i \cup \{i\}$ and $|X \cap S_i| \leq |S_i|$,

$$|(\tilde{f}_{S_i, i}(X)) - \tilde{f}_{S_i, i}(S_i) - (f_{S_i, i}(X) - f_{S_i, i}(S_i))| \leq \epsilon/(2n)$$

Let opt_i denote the optimal cost that can be achieved by using f in i -th iteration. If the greedy algorithm use f to find the solution has performance at least $(1 - 1/e) \cdot \text{opt}_i$, then using $\tilde{f}$, we can obtain a solution that has performance at least $(1 - 1/e) \cdot \text{opt}_i - \epsilon$

Proof. The proof follows from Lemma D.18. □

Next, we explain how to add items into sets based on exact values.

Definition D.20 (Expanding items based on exact value). If the following conditions hold

- Let $S_i \subseteq [i - 1]$.
- Let $f_{S_i, i} : 2^{[i]} \rightarrow \mathbb{R}$.

Then, we can define S_{i+1} as follows

- If $|S_i| = k$ then $S_{i+1} = S_i \cup \{i\} \setminus u$ where $u = \arg \max_{v \in S_i \cup \{i\}} f_{S_i, i}(S_i \cup \{i\} \setminus v) - f_{S_i, i}(S_i)$
- If $|S_i| < k$, then $S_{i+1} = S_i \cup \{i\}$.